

Kafka Consumer Lag Resolution SOP

Objective

Ensure timely processing of Kafka messages by identifying and resolving consumer lag before it impacts downstream processing or SLA commitments.

Scope

Applicable to Kafka-based applications, streaming platforms, backend services, data engineering pipelines, DevOps teams, and platform teams supporting production or synthetic incident workflows.

Pre-requisites

1. Access to Kafka cluster, namespace or broker host.
2. Access to monitoring dashboards such as Grafana, Prometheus, or cloud monitoring.
3. Kafka CLI tools: kafka-topics, kafka-consumer-groups, kafka-console-producer, and kafka-console-consumer.
4. Permission to view application logs, broker logs, and deployment status.
5. Escalation contact for DevOps, platform, and application owner teams.

Incident Metadata

Field	Value
Ticket ID	T005
Issue	Kafka consumer lag increasing
Category	Streaming
Service	Kafka
Severity	High
Status	Resolved
Description	Messages are not consumed fast enough
Expected Resolution	Increase partitions and scale consumers

Related Log Evidence

Log ID	Timestamp	Level	Message	Error Code
L004	5/1/2026 10:02	ERROR	Consumer lag exceeded threshold	KAFKA_LAG
L023	5/1/2026 10:11	WARNING	Consumer lag exceeded threshold	KAFKA_LAG
L039	5/1/2026 10:19	ERROR	Consumer lag exceeded threshold	KAFKA_LAG

Procedure

1. Confirm alert and identify the consumer group, topic, and partition where lag is increasing.
2. Run kafka-consumer-groups to capture current offset, log end offset, and lag.
3. Review application logs for slow processing, retry storms, database latency, or blocked threads.
4. Validate consumer pod or process health, CPU, memory, and restart count.
5. Scale consumer instances only up to the partition count and avoid unnecessary over-scaling.
6. Check topic partition count and confirm whether partitioning strategy supports required throughput.
7. Tune max.poll.records, max.poll.interval.ms, session.timeout.ms, and fetch settings after validating application behavior.
8. Monitor lag recovery until it reaches the expected threshold and update the ticket with evidence.

Common Commands

```
kafka-topics.sh --bootstrap-server <broker>:9092 --describe
kafka-consumer-groups.sh --bootstrap-server <broker>:9092 --describe --group <group>
kafka-broker-api-versions.sh --bootstrap-server <broker>:9092
kubectl get pods -n <namespace> | grep kafka
kubectl logs <kafka-pod> -n <namespace> --tail=100
```

Troubleshooting Matrix

Issue	Possible Cause	Resolution
Lag increasing	Slow consumer or downstream dependency	Scale consumers and fix slow dependency
Lag stuck on one partition	Hot key or skewed partition	Review partition key and rebalance workload
Consumer restarts	Memory or processing failure	Check logs and resource limits
Lag returns after recovery	Throughput mismatch	Tune configs or increase partitions

Best Practices

1. Create consumer lag alerts by group, topic, and partition.
2. Do not scale consumers beyond topic partition count.
3. Avoid blocking operations inside message processing.
4. Track processing time and commit latency.

References

Apache Kafka Docs | Internal Logs CSV | Tickets CSV | Monitoring Dashboards | Kafka CLI Runbook